



UNIVERSIDADE DO VALE DO ITAJAÍ
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA DE SISTEMAS OPERACIONAIS

THIAGO WYSE DOS SANTOS
VITOR HUGO REIS DE JESUS
MATHEUS SOUZA BERTEMES

**SISTEMA DE PROCESSAMENTO PARALELO DE REQUISIÇÕES A UM
BANCO DE DADOS UTILIZANDO IPC, THREADS E EXCLUSÃO MÚTUA**

Itajaí, SC - Brasil
Setembro, 2026

Sistema de Processamento Paralelo de Requisições a um Banco de Dados Utilizando IPC, Threads e Exclusão Mútua

Thiago Wyse dos Santos; Vitor Hugo Reis de Jesus; Matheus Souza Bertemes

Universidade do Vale do Itajaí - UNIVALI

Disciplina: Sistemas Operacionais

Repositório: https://github.com/thiagowyseoficial/m1_processamento_paralelo_requisicoes_bd

Resumo

Este trabalho apresenta o desenvolvimento de um sistema cliente-servidor em C++ para simular o processamento de requisições a um banco de dados. O cliente e o servidor são executados como processos separados e se comunicam por meio de um Named Pipe da API Win32. O servidor utiliza um conjunto de threads para processar as requisições recebidas e uma fila compartilhada para organizar as tarefas. A estrutura que representa o banco de dados é compartilhada entre as threads e protegida por um semáforo binário. Esse mecanismo permite que apenas uma thread acesse a região crítica por vez, reduzindo o risco de condições de corrida e inconsistência dos dados. O sistema permite operações de inserção, consulta, atualização e exclusão de registros, além da persistência dos dados em arquivo JSON e do registro das operações em arquivo de log. Também foram realizados testes com diferentes quantidades de threads. Os resultados mostraram redução no tempo médio de execução ao utilizar duas e quatro threads em comparação com uma única thread. Entretanto, o ganho não ocorreu na mesma proporção que o aumento do número de threads, pois parte do processamento precisa ser realizada de forma exclusiva devido ao uso do semáforo.

Palavras-chave: sistemas operacionais; IPC; threads; concorrência; exclusão mútua; Named Pipe.

SUMÁRIO

Resumo	2
1. Introdução	4
2. Objetivos	4
3. Fundamentação Teórica	5
3.1 Comunicação entre processos.....	5
3.2 Threads	5
3.3 Fila de requisições	5
3.4 Exclusão mútua	5
3.5 Sincronização e desempenho.....	6
4. Arquitetura da Solução.....	6
5. Implementação.....	7
5.1 Processo cliente	7
5.2 Comunicação por Named Pipe.....	7
5.3 Processo servidor e pool de threads.....	7
5.4 Fila de requisições	8
5.5 Controle da região crítica.....	8
5.6 Estrutura do banco e operações CRUD.....	8
5.7 Persistência e log	8
6. Fluxo de uma Requisição.....	9
7. Metodologia dos Testes	9
8. Testes Funcionais	10
9. Resultados de Desempenho	10
10. Análise dos Resultados	11
11. Limitações dos Testes.....	12
12. Conclusão	12
Referências.....	13

1. Introdução

Sistemas operacionais oferecem recursos para permitir que processos e threads trabalhem de forma coordenada. Entre esses recursos estão os mecanismos de comunicação entre processos, conhecidos como IPC, e os mecanismos de sincronização usados quando várias threads acessam dados compartilhados.

Neste trabalho foi desenvolvido um sistema que simula o funcionamento básico de um gerenciador de requisições a um banco de dados. A aplicação foi dividida em dois processos: um cliente e um servidor. O cliente envia comandos ao servidor, enquanto o servidor recebe essas requisições e distribui o processamento entre várias threads.

A comunicação entre cliente e servidor foi implementada por meio de Named Pipe, um recurso da API Win32 para comunicação entre processos. No servidor, as requisições recebidas são armazenadas em uma fila e processadas por um conjunto fixo de threads.

Como as threads utilizam a mesma estrutura de dados, é necessário controlar o acesso à região compartilhada. Para isso, foi utilizado um semáforo binário. Dessa forma, apenas uma thread pode acessar a estrutura protegida por vez.

O sistema permite realizar as operações INSERT, SELECT, UPDATE e DELETE. Os registros são mantidos em memória por meio de um vector<Registro> e persistidos no arquivo banco.json. O servidor também registra informações sobre as operações executadas em um arquivo de log.

Além da implementação funcional, foram realizados testes com diferentes quantidades de threads. O objetivo foi observar o comportamento do sistema e comparar o tempo de execução e a quantidade de operações processadas por segundo.

2. Objetivos

O objetivo geral deste trabalho é desenvolver uma aplicação que demonstre, de forma prática, os conceitos de comunicação entre processos, concorrência, paralelismo e exclusão mútua estudados na disciplina de Sistemas Operacionais.

Como objetivos específicos, o sistema busca manter cliente e servidor como processos separados, utilizar um mecanismo real de IPC, distribuir as requisições entre várias threads, controlar o acesso à estrutura compartilhada e permitir a execução das quatro operações básicas do banco de dados.

Também faz parte do trabalho analisar o comportamento da aplicação com diferentes quantidades de threads e verificar como a sincronização da região crítica influencia o desempenho do sistema.

3. Fundamentação Teórica

3.1 Comunicação entre processos

A comunicação entre processos permite que dois ou mais processos troquem informações mesmo possuindo espaços de memória separados. Os sistemas operacionais oferecem diferentes formas de IPC, como pipes, filas de mensagens, memória compartilhada e sockets.

Neste projeto foi utilizado um Named Pipe da API Win32. O servidor cria o canal de comunicação e espera a conexão do cliente. Após a conexão, o cliente envia uma requisição e recebe a resposta pelo mesmo canal. A implementação utiliza funções como `CreateNamedPipeW`, `ConnectNamedPipe`, `CreateFileW`, `ReadFile` e `WriteFile`.

3.2 Threads

Uma thread representa uma linha de execução dentro de um processo. Diferentes threads do mesmo processo podem executar tarefas de forma concorrente e compartilhar estruturas de memória. No sistema desenvolvido, o servidor cria um conjunto de threads no momento de sua inicialização. Em vez de criar uma nova thread para cada requisição, o servidor utiliza um pool de threads, mantendo as workers ativas e reutilizando-as conforme as tarefas chegam.

3.3 Fila de requisições

As requisições recebidas pelo servidor são armazenadas em uma fila compartilhada. Como várias threads podem acessar essa fila, é necessário protegê-la com `std::mutex`. Também é utilizada uma `std::condition_variable`, que permite que as workers aguardem novas tarefas sem permanecer executando verificações continuamente.

3.4 Exclusão mútua

Quando diferentes threads acessam a mesma estrutura de dados, podem ocorrer problemas caso duas ou mais tentem modificar os mesmos dados ao mesmo tempo. Esse tipo de situação pode causar uma condição de corrida. Para evitar esse problema, são

utilizados mecanismos de exclusão mútua. No sistema desenvolvido, o banco de dados e o arquivo de log são protegidos por um semáforo criado com contador inicial e máximo igual a um:

```
CreateSemaphoreW(nullptr, 1, 1, nullptr);
```

Antes de acessar a região protegida, a thread utiliza WaitForSingleObject. Quando termina o processamento, utiliza ReleaseSemaphore. Na prática, esse semáforo funciona como um mecanismo de exclusão mútua, pois apenas uma thread pode permanecer na região crítica por vez.

3.5 Sincronização e desempenho

O uso de várias threads pode reduzir o tempo de execução quando diferentes tarefas podem ser executadas ao mesmo tempo. Entretanto, a sincronização também pode limitar esse ganho. Estudos sobre seções críticas em programas com múltiplas threads mostram que a contenção na região protegida reduz parte do benefício esperado com o paralelismo. Esse comportamento também foi observado neste trabalho, no qual o ganho de desempenho não cresceu na mesma proporção que o número de threads.

4. Arquitetura da Solução

A aplicação foi organizada em dois processos principais: o cliente e o servidor. O cliente recebe os comandos do usuário e os envia ao servidor. O servidor recebe as requisições, organiza as tarefas em uma fila e distribui o processamento entre as threads disponíveis. O acesso ao banco de dados compartilhado é protegido por um semáforo binário. Além do banco mantido em memória, as alterações são persistidas em banco.json, enquanto as operações são registradas em log.txt.

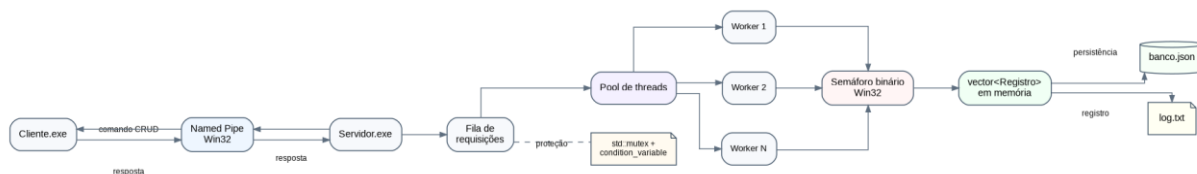


Figura 1 - Arquitetura geral do sistema.

Fonte: elaborada pelos autores.

5. Implementação

5.1 Processo cliente

O cliente foi implementado no arquivo `Cliente.cpp`. Ele pode ser utilizado de forma interativa ou por meio de argumentos na linha de comando. Ao receber uma operação destinada ao servidor, o cliente estabelece uma conexão com o Named Pipe, envia o comando e aguarda a resposta.

```
bool executar(const string& comando) {  
    auto pipe = ipc::conectar();  
    ipc::enviar(pipe.get(), comando);  
    const string resposta = ipc::receber(pipe.get());  
    cout << resposta << '\n';  
    return resposta.rfind("OK:", 0) == 0;  
}
```

5.2 Comunicação por Named Pipe

As funções responsáveis pela comunicação entre os processos estão nos arquivos `IPC.hpp` e `IPC.cpp`. O servidor cria o Named Pipe com `CreateNamedPipeW`. O pipe é duplex, permitindo leitura e escrita pelos dois processos, e utiliza modo de mensagem com operações bloqueantes. O cliente se conecta com `CreateFileW`. Depois da conexão, `WriteFile` e `ReadFile` são usados para transmitir as mensagens. Cada mensagem possui limite de 4096 bytes, e o servidor utiliza `FlushFileBuffers` antes de encerrar a conexão.

5.3 Processo servidor e pool de threads

O servidor foi implementado no arquivo `Servidor.cpp`. Ao ser iniciado, ele permite informar a quantidade de threads que serão utilizadas, com valores entre 1 e 16. Caso nenhum valor seja informado, são utilizadas quatro threads. As threads são criadas uma única vez e permanecem ativas durante a execução do servidor, formando um pool fixo.

```
for (int i = 1; i <= quantidade; ++i) {  
    workers.emplace_back(  
        processar, ref(fila), ref(banco), ref(log),  
        semaforo.get(), i);  
}
```

5.4 Fila de requisições

O servidor utiliza uma `queue<Tarefa>` para armazenar as requisições recebidas. Como essa fila pode ser acessada por diferentes threads, sua estrutura é protegida por

std::mutex. Quando não existem tarefas disponíveis, std::condition_variable suspende a worker até que uma nova tarefa seja adicionada ou o servidor seja encerrado.

5.5 Controle da região crítica

O banco compartilhado e o log são protegidos por um semáforo binário. O contador inicial e o contador máximo são iguais a um:

```
ipc::Handle semaforo(CreateSemaphoreW(nullptr, 1, 1, nullptr));
```

Antes de acessar a região protegida, a worker chama WaitForSingleObject. Ao final da operação, ReleaseSemaphore libera o recurso para outra thread. A implementação utiliza uma classe auxiliar para garantir que a liberação ocorra quando o objeto sai do escopo, inclusive em caso de erro.

5.6 Estrutura do banco e operações CRUD

O banco é representado em memória por um vector de registros. Cada registro possui um identificador inteiro e um nome:

```
struct Registro {  
    int id;  
    string nome;  
};
```

O sistema implementa INSERT, SELECT, UPDATE e DELETE. INSERT adiciona um registro quando o identificador ainda não existe. SELECT consulta um registro pelo identificador. UPDATE altera o nome associado ao registro. DELETE remove o registro. A função interpretar() também valida a entrada, exigindo ID inteiro positivo e os parâmetros necessários para cada operação.

5.7 Persistência e log

As alterações são persistidas no arquivo banco.json. Antes de substituir o arquivo principal, a aplicação grava os dados em banco.json.tmp. Depois da gravação, o arquivo temporário substitui o banco anterior. Essa estratégia reduz o risco de deixar o arquivo principal incompleto em caso de falha durante a escrita. O servidor também mantém o arquivo log.txt, que registra data, horário, número da worker, identificador da thread, comando recebido, resposta e tempo de processamento em microssegundos.

6. Fluxo de uma Requisição

O processamento começa quando o usuário digita um comando no cliente. O comando é enviado ao servidor pelo Named Pipe e colocado na fila de tarefas. Uma worker retira a requisição, interpreta o comando e solicita acesso ao semáforo. Após obter acesso, executa a operação sobre a tabela compartilhada. Quando necessário, os dados são persistidos em banco.json e a operação é registrada no log. Por fim, o semáforo é liberado e a resposta retorna ao cliente pelo Named Pipe.

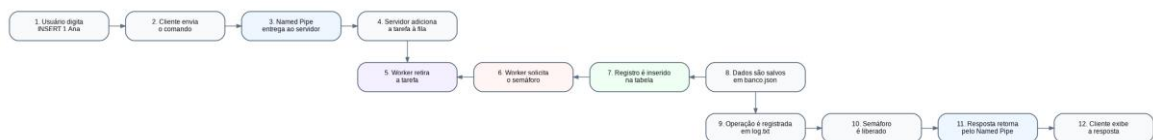


Figura 2 - Sequência de processamento de uma requisição INSERT.

Fonte: elaborada pelos autores.

7. Metodologia dos Testes

Os experimentos foram realizados em 19 de setembro de 2026, em ambiente Windows, com o projeto compilado em C++17 utilizando MSVC, otimização /O2 e avisos tratados como erro com /W4 /WX.

Característica	Ambiente observado
Sistema	Windows
Compilação	MSVC / C++17 / O2
Clientes por rodada	4 processos simultâneos
Pool avaliado	1, 2 e 4 threads
Repetições	3 por configuração
Operações por rodada	400
Total das medições	3.600 operações

Cada cliente executou 25 ciclos de INSERT, SELECT, UPDATE e DELETE, totalizando 100 comandos por cliente. Para evitar disputa por identificador durante o teste de desempenho, cada cliente utilizou uma faixa própria de IDs. O tempo foi medido imediatamente antes de iniciar os quatro clientes e terminou após o encerramento do último cliente. A medição incluiu criação dos clientes, IPC, operações CRUD, persistência, log e saída dos processos. A preparação das entradas, a inicialização do servidor e o seu encerramento ficaram fora do tempo medido.

8. Testes Funcionais

Antes da comparação de desempenho, foram realizados testes para verificar as principais operações, as validações de entrada, a persistência e o comportamento do sistema em situações de disputa pelo mesmo identificador.

Teste executado	Resultado observado
INSERT e SELECT	Registro inserido e consultado corretamente
UPDATE e DELETE	Alteração e exclusão persistidas
ID duplicado	Operação rejeitada
Consulta de ID excluído	Registro não encontrado
ID não numérico e UPDATE sem nome	Comandos rejeitados com mensagem
Nome com acento, aspas e barra	Transporte e gravação corretos
Encerrar e reiniciar	Registro recuperado do JSON
12 clientes inserindo ID 99	1 inserção aceita e 11 rejeições por duplicidade
9 rodadas com quatro clientes	3.600 respostas OK e bancos finais vazios

O teste com 12 clientes tentando inserir o mesmo identificador verificou um caso relevante de acesso concorrente. Apenas uma inserção foi aceita, enquanto as outras foram rejeitadas por duplicidade. Esse resultado é compatível com a exclusão mútua esperada para a região protegida.

9. Resultados de Desempenho

Threads	Repetição 1 (ms)	Repetição 2 (ms)	Repetição 3 (ms)
1	840,238	880,599	979,613
2	627,714	579,892	663,110
4	582,064	547,057	593,386

Threads	Média (ms)	Desvio-padrão (ms)	Vazão (op/s)	Ganho relativo
1	900,150	71,715	444,37	1,00
2	623,572	41,763	641,47	1,44
4	574,169	24,152	696,66	1,57

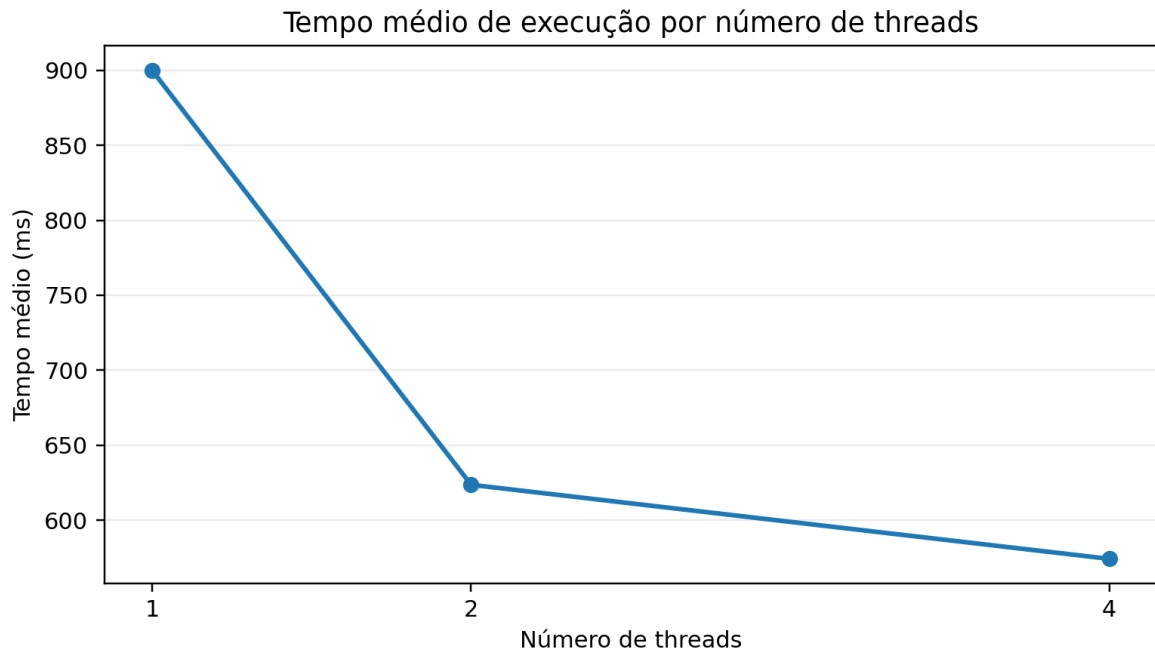


Figura 3 - Tempo médio de execução por número de threads.

Fonte: elaborada pelos autores.

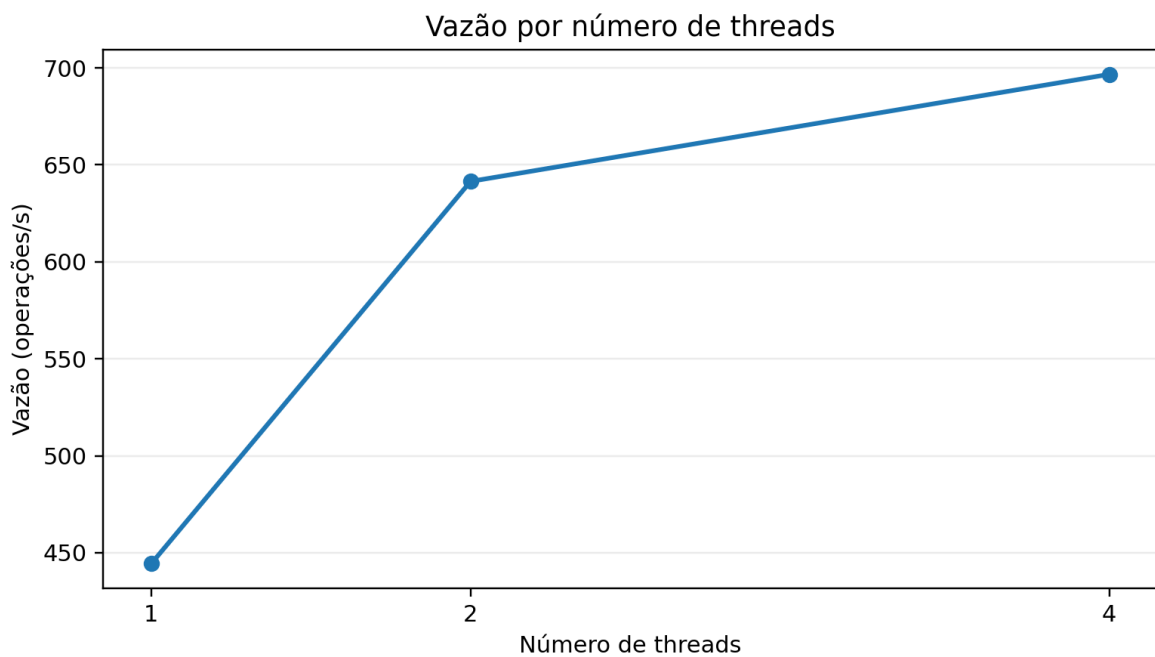


Figura 4 - Vazão por número de threads.

Fonte: elaborada pelos autores.

10. Análise dos Resultados

Os resultados mostram que o uso de mais threads reduziu o tempo médio de execução no cenário testado. Com uma thread, o tempo médio foi de 900,150 ms. Com

duas threads, caiu para 623,572 ms. Com quatro threads, o menor tempo médio foi de 574,169 ms.

Em comparação com uma thread, a configuração com quatro threads reduziu o tempo médio em aproximadamente 36,2%. A vazão aumentou de 444,37 operações por segundo para 696,66 operações por segundo. O ganho relativo chegou a 1,57.

Mesmo com a melhora, o ganho não foi proporcional ao número de threads. Isso ocorre porque o acesso ao banco, a persistência e o registro da operação passam por uma região protegida pelo semáforo. Portanto, existe concorrência entre as workers, mas parte do trabalho continua sendo executada de forma exclusiva.

Dessa forma, não é correto afirmar que quatro threads serão sempre melhores em qualquer situação. O que os resultados mostram é que, neste cenário específico, a configuração com quatro threads apresentou o melhor desempenho entre as opções avaliadas.

11. Limitações dos Testes

Os resultados representam apenas o ambiente e a carga utilizados durante os experimentos. Foram feitas três repetições por configuração, o que permite uma comparação inicial, mas não cobre todos os cenários possíveis. Fatores externos, como uso do processador por outros programas, cache, escalonamento das threads e operações de escrita no disco, podem alterar os tempos observados.

12. Conclusão

Este trabalho apresentou o desenvolvimento de um sistema cliente-servidor em C++ para demonstrar, na prática, conceitos de comunicação entre processos, threads, concorrência e exclusão mútua. O cliente e o servidor foram implementados como processos separados e se comunicam por meio de um Named Pipe da API Win32. No servidor, as requisições são organizadas em uma fila e processadas por um conjunto de threads.

O acesso ao banco de dados compartilhado é protegido por um semáforo binário. Dessa forma, apenas uma thread pode acessar a região crítica por vez, reduzindo o risco de condições de corrida e inconsistência dos dados. O sistema implementa as operações INSERT, SELECT, UPDATE e DELETE, além de persistir os registros em arquivo JSON e manter um arquivo de log com informações sobre as operações realizadas.

Os testes funcionais mostraram que as operações foram executadas corretamente, inclusive em situações de disputa pelo mesmo identificador. Nos testes de desempenho, o uso de duas e quatro threads reduziu o tempo médio de execução em comparação com uma única thread. A configuração com quatro threads apresentou o menor tempo médio, porém o ganho não foi proporcional ao aumento do número de threads, devido à presença da região crítica protegida.

Referências

1. IMMICH, Patricia K.; BHAGAVATULA, Ravi; PENDSE, Ravi. Performance analysis of five interprocess communication mechanisms across UNIX operating systems. *Journal of Systems and Software*, v. 68, n. 1, p. 27-43, 2003. DOI: 10.1016/S0164-1212(02)00134-6.
2. KAGARIS, Dimitri; DUTTA, Sourav; EYERMAN, Stijn. Execution Time Estimation of Multithreaded Programs With Critical Sections. *IEEE Transactions on Parallel and Distributed Systems*, v. 33, n. 10, p. 2470-2481, 2022. DOI: 10.1109/TPDS.2022.3143455.
3. EYERMAN, Stijn; EECKHOUT, Lieven. Modeling critical sections in Amdahl's law and its implications for multicore design. *Proceedings of the 37th Annual International Symposium on Computer Architecture*, p. 362-370, 2010. DOI: 10.1145/1815961.1816011.
4. MICROSOFT. CreateNamedPipeW function. Microsoft Learn. Documentação da API Win32.
5. MICROSOFT. ConnectNamedPipe function. Microsoft Learn. Documentação da API Win32.
6. MICROSOFT. CreateSemaphoreW function. Microsoft Learn. Documentação da API Win32.
7. MICROSOFT. WaitForSingleObject function. Microsoft Learn. Documentação da API Win32.
8. MICROSOFT. ReleaseSemaphore function. Microsoft Learn. Documentação da API Win32.
9. UNIVERSIDADE DO VALE DO ITAJAÍ. Avaliação M1 – IPC, Threads e Paralelismo. *Disciplina de Sistemas Operacionais*, 2026.